UNIVERSITY OF INFORMATION TECHNOLOGY
FACULTY OF INFORMATION SYSTEMS

PROCEDURAL LANGUAGE / STRUCTURED QUERY LANGUAGE (PL/SQL)

Dr. Cao Thi Nhan

Content

1. Introduction PL/SQL
2. PL/SQL Block
3. Constants, variables, Data types
4. Condition (If, case); loop
5. Exception
6. Cursors
7. Function, Procedure, Trigger, Package

Introduction

- PL/SQL:
  - a procedural programming language of Oracle
  - a combination of SQL and control structures, functions, cursor, exception, and transaction statements.
  - allows the use of all data manipulation commands including INSERT, DELETE, UPDATE and SELECT, COMMIT, ROLLBACK, SAVEPOINT, control structures such as loops (for, while, loop), if/else,...
  - Using PL/SQL to create units (such as procedures, functions, packages, types, and triggers) stored in the database for reuse by applications

PL/SQL Block

- PL/SQL statements are divided into logical blocks and nested blocks. Variables can be declared locally within the block and exception handling is handled in the block where the error occurs
- A block consists of three parts: the declaration part (where variables are declared), the execution part, and the exception handling part (error conditions or warnings).

PL/SQL Block


```
DECLARE    /*Block 1*/                                   --- Block 1
        Declarations of block 1
BEGIN
        Executable Statements
        DECLARE    /*Block 2*/                           --- Block 2
            Declarations of block 2
        BEGIN
            Executable Statements
        EXCEPTION
            Exception Handlers
        END;                                             --- End Block 2
    EXCEPTION
        Exception Handlers of block 1
    END;                                                 --- End Block 1
```

PL/SQL Block

- Function, Procedure are named PL/SQL block. They can be stored in the database and reuse.
- Block without name called anonymous block

Constants, variables, Data types

- Data types:
  - Number
  - Boolean: used in if-then, case structure
  - Character: char (fixed length), varchar2 (varying length), long, raw…
  - Datetime
- https://docs.oracle.com/cd/B28359_01/server.111/b28318/datatype.htm#i16209

Constants, variables, Data types

- Constants: stores a constant value
  - Syntax: constant_name CONSTANT datatype := constant_value
  - ratio CONSTANT NUMBER(3,2) := 1.86;
- Variables: stores a value of a data type
  - Syntax: variable_name datatype [NOT NULL] [:= initial_value]
  - Emp_name VARCHAR2 (50) := 'Nguyen Van Minh';
  - Product_name VARCHAR2 (70) := 'Keyboard';
  - Salary NUMBER(5);

Constants, variables, Data types

- Constants: stores a constant value
  - Syntax: constant_name CONSTANT datatype := constant_value
  - ratio CONSTANT NUMBER(3,2) := 1.86;
- Variables: stores a value of a data type
  - Syntax: variable_name datatype [NOT NULL] [:= initial_value]
  - Emp_name VARCHAR2 (50) := 'Nguyen Van Minh';
  - Product_name VARCHAR2 (70) := 'Keyboard';
  - Salary NUMBER(5);

Constants, variables, Data types

- %TYPE

```
declare
    x emp.empno%type;
    y emp.ename%type;
begin
  select empno, ename into x,y from emp where empno='7369';
  dbms_output.put_line(‘Emp ID:' || x || ‘ – Emp Name:' || y);
end;
```

Constants, variables, Data types

- %ROWTYPE Used to declare a variable that references a row in a datatable.
  - v_EmpID has the same data type as the EmpID column in the EMPLOYEE table

    v_Emp EMPLOYEE%ROWTYPE
  - To access column, use the Syntax: Variable-name.Column-name

    v_Emp.ID, v_Emp.Name

Constants, variables, Data types

- %ROWTYPE

```
declare
    z       emp%rowtype;
begin
    select * into z from emp where empno='7369';
    dbms_output.put_line('Emp ID:' || z.empno || ' – EMP
    Name:' || z.ename);
end;
```

If – then statements

- If .. then, if .. then .. else, if .. then .. elsif

```
IF <condition> THEN
        statements;
END IF;
```

```
declare
   min_salary NUMBER := 5000;
begin
   if min_salary > 3000 then
    dbms_output.put_line('Min salary is higher than 3000');
   end if;
end;
```

If – then statements

- If .. then, if .. then .. else, if .. then .. elsif

```
IF < condition 1> THEN
    statements 1;
ELSE
    IF < condition 2> THEN
        statements 2;
    ELSE
        .....;
    END IF;
END IF;
```

```
declare
  n NUMBER; result VARCHAR2(50);
begin
  if n = 0 then result:='Negative'
  else if n = 1 then result:='Positive';
      end if;
  end if;
  dbms_output.put_line('The result is' ||
  result);
end;
```

If – then statements

- If .. then, if .. then .. else, if .. then .. elsif

```
IF < condition 1> THEN
      statements 1;
   ELSIF < condition 2> THEN
            statements 2;
   ELSIF < condition 3> THEN
            statements 3;
END IF;
```

```
declare
   n NUMBER; result VARCHAR2(50);
begin
   if n = 0 then result:='Negative'
   elsif n = 1 then result:='Positive';
end if;
   dbms_output.put_line('The result is' ||
   result);
end;
```

Case statements


```
CASE  [ expression ]
   WHEN condition_1 THEN result_1
   WHEN condition_2 THEN result_2
   ...
   WHEN condition_n THEN result_n
   ELSE result
END
```

```
select    table_name,
   TABLESPACE_NAME, CASE
   owner
    WHEN 'SYS'            THEN
        'The owner is SYS'
    WHEN 'SYSTEM'     THEN
        'The owner is SYSTEM'
    ELSE 'The owner is another value'
   END     as Owner
   from all_tables;
```

Case statements


```
CASE  [ expression ]
   WHEN condition_1 THEN result_1
   WHEN condition_2 THEN result_2
   ...
   WHEN condition_n THEN result_n
   ELSE result
END
```

```
select    table_name,
   TABLESPACE_NAME, CASE
      WHEN owner = 'SYS'
        THEN          'The owner is SYS'
      WHEN owner = 'SYSTEM'
   THEN       'The owner is SYSTEM'
      ELSE 'The owner is another value'
      END     as Owner
from all_tables;
```

Loop statements


```
LOOP
    <statements>
    IF <stop_condition> THEN
        ....
        EXIT;
    END IF;
    END LOOP;
```

```
declare
    z  number :=1;
    BEGIN
        LOOP
        z :=z+5;
        IF (z>=100) THEN
            exit;
        End IF;
        END LOOP;
    END;
```

For … Loop statements

- Case 1:

```
FOR v IN [REVERSE] start_value .. end_value
        LOOP
            <statements>
        END LOOP;
```

- Case 2:

```
FOR v IN (select statement)
        LOOP
        < statements > (… v.column1, v.column2 …)
        END LOOP;
```

For ... Loop statements

- Case 1:

```
FOR v IN [REVERSE] start_value .. end_value
        LOOP
            <statements>
        END LOOP;
```

```
declare
    z  number:=1; i number;
    BEGIN
        FOR i IN 1 .. 10 LOOP
            z :=z+3;
        END LOOP;
END;
```

For … Loop statements

- Case 2:

```
FOR v IN (select statement)
        LOOP
        < statements > /*(… v.column1, v.column2 …) */
        END LOOP;
```

```
for z in (select emp.empno, emp.ename from emp)
    loop
        Dbms_output.put_line (z.empno || '---' || z.ename);
    end loop;
```

while ... Loop statements


```
WHILE <condition>
        LOOP
                < statements>
        END LOOP;
-------------------
declare
        z  number:=1; i   number:=1;
        BEGIN
                WHILE (i<=10)
                    LOOP
                      i:=i+1; z :=z+3;
                    END LOOP;
        END;
```

GOTO statements

- GOTO label_name
- Label_name: <<label_name>>
- Can not GOTO a label which stay in IF, CASE, LOOP statement

```
BEGIN
    Dbms_output.put_line ('Goto Test');
    GOTO Block1;
    <Block1> Dbms_output.put_line ('Good morning'); GOTO Block3;
    <Block3> Dbms_output.put_line ('Good afternoon'); GOTO Block2;
    <Block2> Dbms_output.put_line ('Good night');
END;
```

EXCEPTION

- When an error occurs, an exception is raised, the program execution is stopped, and control is transferred to the PL/SQL block containing the exception handler.
- Implicit exceptions are raised, whereas user-defined exceptions are raised explicitly using the RAISE statement.
- Predefined exceptions Example: **ZERO_DIVIDE:** A program attempts to divide a number by zero.

https://docs.oracle.com/cd/B14117_01/appdev.101/b10807/07_errs.htm

EXCEPTION

https://docs.oracle.com/cd/B14117_01/appdev.101/b10807/07_errs.htm

Predefined exception	**Raised when ...**
CURSOR_ALREADY_OPEN	A program attempts to open an already open cursor.
NO_DATA_FOUND	A SELECT INTO statement returns no rows, or your program references a deleted element in a nested table or an uninitialized element in an index-by table.
TOO_MANY_ROWS	A SELECT INTO statement returns more than one row.
ZERO_DIVIDE	A program attempts to divide a number by zero.
STORAGE_ERROR	PL/SQL runs out of memory or memory has been corrupted.
.....	

EXCEPTION


```
DECLARE
        sName Student.Name%TYPE;
        StudentId number;
BEGIN
        StudentId := &abc;
        select Name into sName from Student where ID = StudentId;
        dbms_output.put_line(sName);
EXCEPTION
        WHEN NO_DATA_FOUND then
        dbms_output.put_line ('This student does not exist.');

END;
```

EXCEPTION

- User defines exceptions

```
DECLARE
/*if Block of Function or Procedure then do not use Declare*/
        Exception_Name  EXCEPTION;
        ....
BEGIN
        ...
        IF <error> then
            RAISE Exception_Name ;
        END IF  ;
EXCEPTION
    WHEN Exception_Name then
        .....
    WHEN OTHERS then
        ....
END;
```

EXCEPTION

- User defines exceptions

```
Create Function CheckStudentID (ID number) return number  As
        Dup_St_ID  EXCEPTION;
BEGIN
        IF ID=100 then
          RAISE Dup_St_ID ;
        ELSE
          return 2;
        END IF  ;
EXCEPTION
      WHEN Dup_St_ID then
        return 1;
      WHEN OTHERS then
        return 0;
END;
----------
Select CheckStudentID (100) from Dual;
Select CheckStudentID (5) from Dual;
```

Relational data model example

1. **EMPLOYEE** (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)
2. **DEPARTMENT** (Dnumber, Dname, Mgr_ssn, Mgr_start_date)
3. **DEPT_LOCATIONS** (Dnumber, Dlocation)
4. **PROJECT** (Pnumber, Pname, Plocation, Dnum)
5. **WORKS_ON** (Essn, Pno, Hours)
6. **DEPENDENT** (Essn, Dependent_name, Sex, Bdate, Relationship)

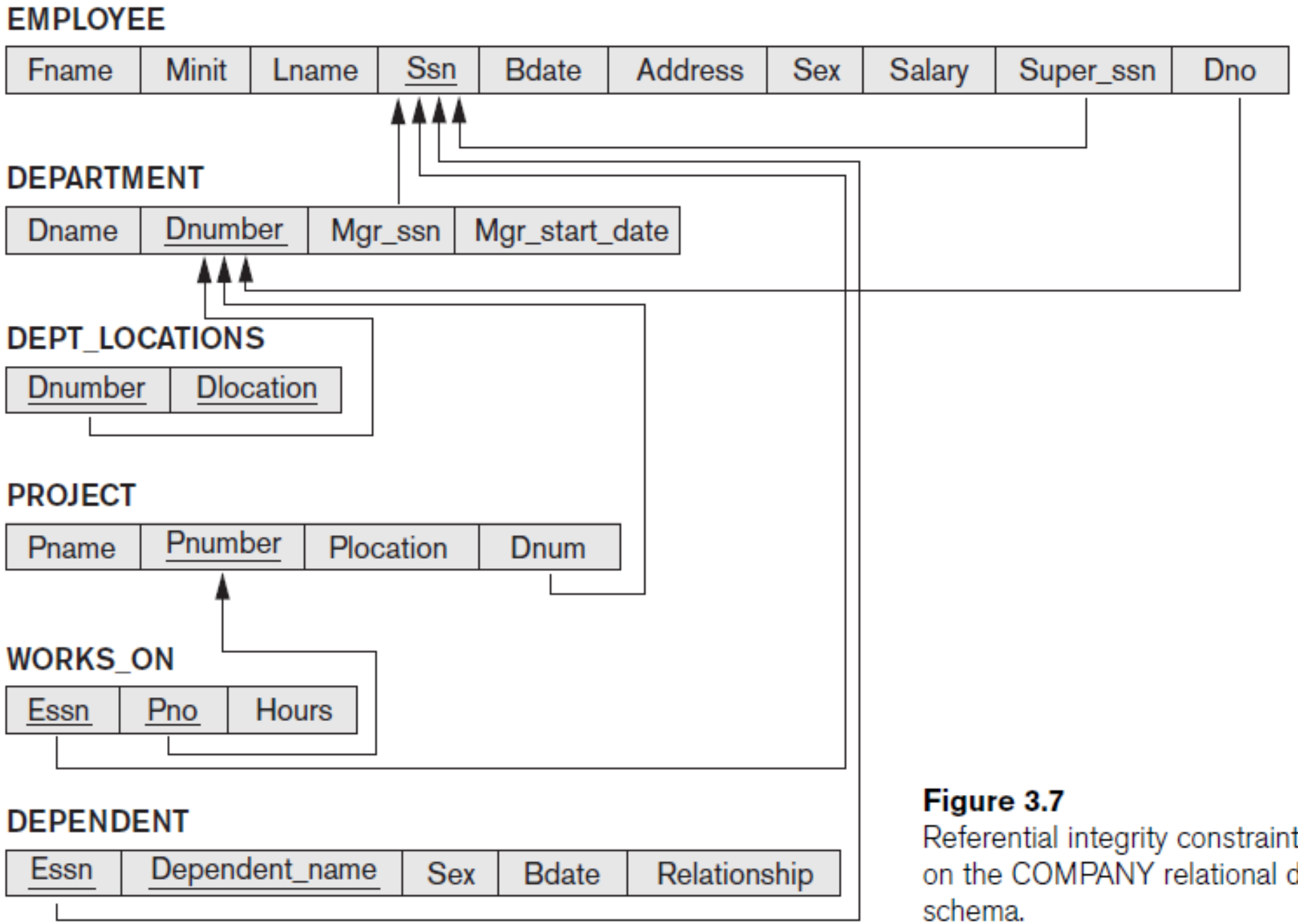

Figure 3.7
Referential integrity constraints displayed on the COMPANY relational database schema.

Q A